ENGINEERING CONTEXT DOCUMENT

NexusOS / Insurance Agent OS

A vertical-integrated operating system for life insurance brokerages. Real-time AI underwriting coaching layered directly into the dialer — not an overlay, not a plugin. Built-in.

BUILD WINDOW	FIRST CLIENT	TARGET MRR	PHASE
3 Weeks to Demo	2,500-agent brokerage	$2.5M	MVP — Demo-ready

01 Project Overview

NexusOS is a **vertically integrated insurance agent operating system**. It is not a generic CRM. It is not GoHighLevel with an insurance skin. It is a purpose-built platform that unifies every workflow a life insurance agent touches — from lead intake to policy bind — with an AI layer that is embedded in the call itself.

CRM Pipeline

GHL-inspired drag-drop stages with auto-triggered SMS/email workflows on stage transitions.

Power Dialer

Browser-based softphone via Twilio Voice SDK. Single-line MVP, parallel dialer deferred for TCPA architecture.

Live Transcription

Deepgram streaming STT with speaker diarization. <2 second latency. Captions render in-call.

AI Underwriting Extraction

Claude tool-use fires every 3 seconds on new transcript chunks. Extracts structured health facts.

Carrier Ranking Engine

Deterministic TypeScript function. Ranks carriers live as health facts emerge. Auditable, E&O-defensible.

Auto-Quote Generator

Post-call quote comparison: top 3 carriers with monthly premiums and the reason each was ranked.

Commission Dashboard

Policy tracking, chargeback alerts, expected vs. paid commission ledger.

Agent Leaderboard

KPIs: calls made, policies written, revenue. Snapshotted per period.

02 The Moat

⚠ **The moat is the AI cockpit.** This is what makes the product defensible. Generic tools like Cluely, Gong, and Chorus overlay Zoom/Meet with general sales suggestions. We are not doing that.

What We Do That Nobody Else Does

- Listens for **medical conditions** during the call and triggers insurance-specific follow-up questions automatically
- Example: client says *"I have diabetes"* → AI coaching panel injects: *"Type 1 or Type 2? A1C level? When diagnosed? Insulin or metformin?"*
- Maintains a **structured underwriting fact panel** that updates in real-time as answers come in
- **Re-ranks carriers live** as new health facts emerge — different carriers have different underwriting tolerances
- Post-call: generates a quote comparison showing top 3 carriers with premiums *and the reason* each was ranked (e.g., "Mutual of Omaha accepts A1C under 7.5 with 3+ years since diagnosis")
- All of this is **deterministic rules, not RAG, not hallucination**. Auditable. E&O-defensible.

WHY RAG WOULD KILL US HERE

If the ranking engine hallucinates a carrier that accepts a condition it doesn't actually accept, and an agent relies on that ranking and submits an app, the brokerage has an Errors & Omissions claim. The ranking logic must be deterministic, structured, and auditable. RAG is reserved for Phase 2+ features like objection handling libraries and historical call search.

03 Market & Business Context

First Client

Life insurance brokerage with **2,500 independent agents**. They currently use LeadChampPro (GHL skin). We have login access.

Pricing Target

~$1,000/agent/month. At 2,500 agents full adoption, that's **$2.5M MRR**.

Phase 2

Replicate for **healthcare insurance**. Same platform skeleton, different carrier APIs and underwriting rules.

→ **Exit thesis:** "The last software before AI fully automates insurance sales." Build the dominant vertical platform now, sell before full automation makes the human agent obsolete.

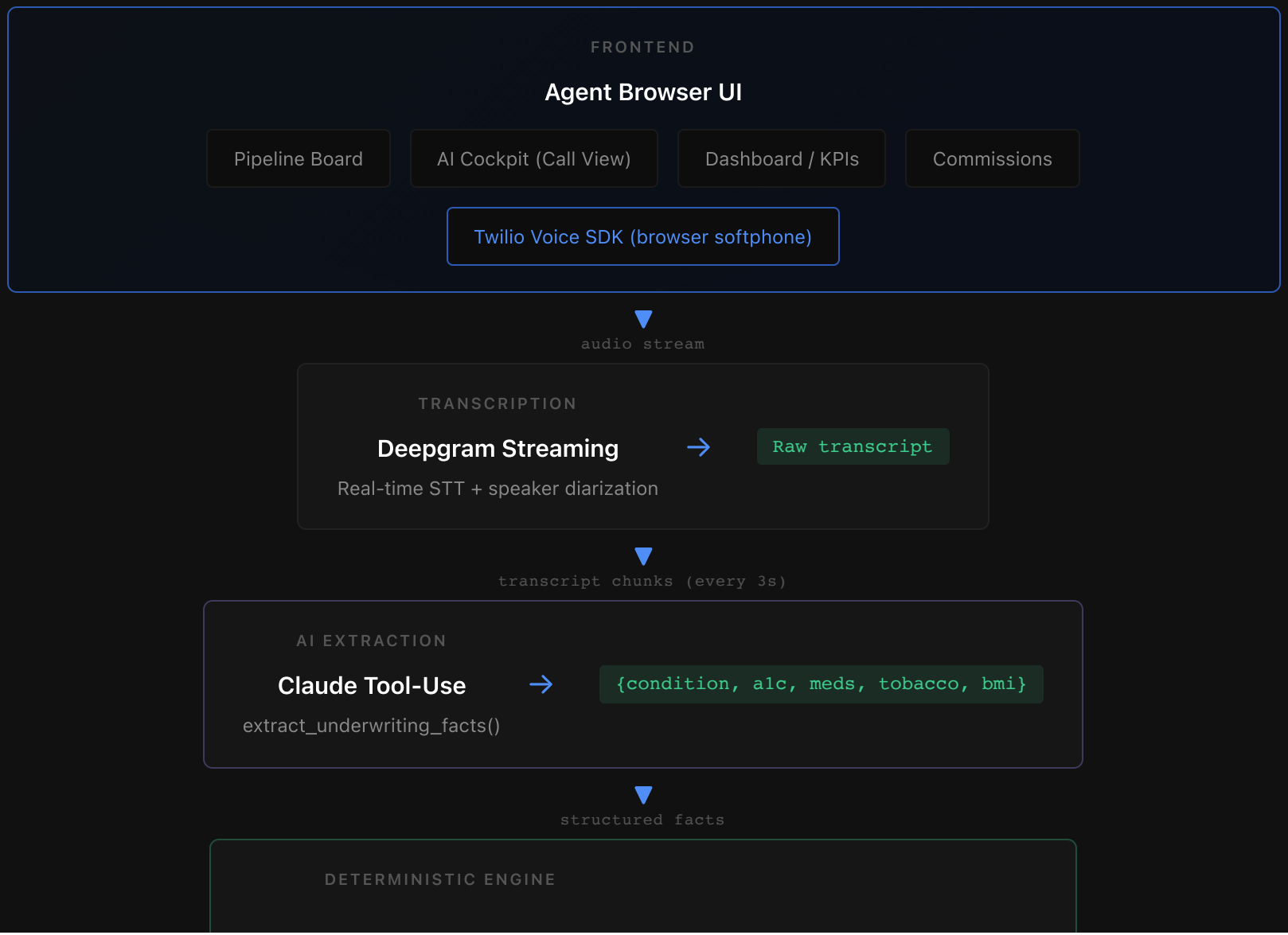
04 Tech Stack

LAYER	TECHNOLOGY	NOTES
Frontend	**Next.js 15 + React 19**	App router, server components throughout
UI	**shadcn/ui + Tailwind CSS v4 + Framer Motion**	Dark theme, LeadChampPro-level polish
Backend / DB	**Supabase**	Postgres + RLS + Edge Functions + Realtime + pgvector. Row-level security per agent/agency.
Auth	**Supabase Auth**	SSO later — Microsoft Entra / Google Workspace. 2,500 agents means SSO is mandatory at scale.
Voice / Dialer	**Twilio Voice SDK**	Browser-based softphone. Single outbound line for MVP. Parallel dialer deferred.
Transcription	**Deepgram Streaming API**	Real-time STT, speaker diarization, <2s latency target
AI Extraction	**Claude API (Sonnet, streaming tool-use)**	`extract_underwriting_facts()` tool, fires every 3s on new transcript chunks
Coaching Engine	**Deterministic rules + Claude**	Rules for underwriting triggers; Claude generates natural language coaching prompts
Carrier Knowledge	**Static JSON → Postgres tables**	Hand-authored underwriting matrices per carrier. Phase 2: real carrier API feeds.
Ranking Engine	**Pure TypeScript function**	Takes extracted facts + coverage amount → ranked carrier list with reasons. Deterministic.
Quote Generator	**Pure TypeScript (MVP) → carrier APIs (Phase 2)**	Static rate formulas for demo. Produces realistic premiums.
Workflows / SMS	**n8n self-hosted OR Supabase Edge Functions + Twilio SMS**	Pipeline stage change → auto-SMS. Choose at kickoff.
Hosting	**VPS + Traefik + Docker**	Existing infra. Alternatively: Vercel (frontend) + Supabase (backend).

⚠ **VPS routing rule:** Traefik is the ONLY reverse proxy. All routes go via docker-compose.yaml Traefik labels. Do not touch or create nginx configs — they are deprecated.

05

Architecture Diagram



Carrier Ranking → `Top 3 carriers + reasons`

TypeScript rules engine — no LLM, no RAG

▼ writes ▼ writes

DATABASE

Supabase (Postgres)

- contacts, policies, pipelines
- transcripts, extracted facts
- carrier rules (JSON → tables)
- commissions, renewals
- RLS per agent / agency
- audit log (append-only)

06 Data Models

All tables live in Supabase (Postgres). Row-level security is enforced at the database level — agents can only read their own contacts; agency admins can read their agents' data; super-admins see everything.

CORE ENTITIES

SQL — supabase/migrations/001_core.sql

```sql
-- Contacts (leads / clients)
CREATE TABLE contacts (
  id                  uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  first_name          text NOT NULL,
  last_name           text NOT NULL,
  phone               text,
  email               text,
  dob                 date,
  state               char(2),
  created_by_agent_id uuid REFERENCES agents(id),
  created_at          timestamptz DEFAULT now()
);

-- Policies
CREATE TABLE policies (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  contact_id      uuid REFERENCES contacts(id),
  carrier_id      uuid REFERENCES carriers(id),
  product_type    text,  -- term_20, whole_life, etc.
  coverage_amount integer,
  premium         numeric(10,2),
  status          text,  -- active, lapsed, chargedback, pending
  effective_date  date,
  expiry_date     date
);

-- Pipeline
CREATE TABLE pipeline_stages (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name              text NOT NULL,
  position          integer,
  auto_sms_template text
);

CREATE TABLE pipeline_cards (
  id         uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  contact_id uuid REFERENCES contacts(id),
  stage_id   uuid REFERENCES pipeline_stages(id),
  agent_id   uuid REFERENCES agents(id),
```

```
  moved_at   timestamptz DEFAULT now()
);
```

VOICE / AI TABLES

SQL supabase/migrations/002_voice_ai.sql

```sql
-- Call sessions
CREATE TABLE call_sessions (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_id        uuid REFERENCES agents(id),
  contact_id      uuid REFERENCES contacts(id),
  twilio_call_sid text,
  started_at      timestamptz,
  ended_at        timestamptz,
  recording_url   text
);

-- Raw transcripts (per utterance)
CREATE TABLE transcripts (
  id               uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  call_session_id  uuid REFERENCES call_sessions(id),
  speaker          text,  -- 'agent' | 'client'
  text             text,
  timestamp_ms     integer
);

-- Extracted underwriting facts
CREATE TABLE extracted_facts (
  id               uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  call_session_id  uuid REFERENCES call_sessions(id),
  contact_id       uuid REFERENCES contacts(id),
  fact_type        text,   -- 'condition', 'tobacco', 'medication', etc.
  fact_key         text,
  fact_value       jsonb,
  confidence       numeric(4,3),
  extracted_at     timestamptz DEFAULT now()
);

-- Live carrier rankings (updated during call)
CREATE TABLE carrier_rankings (
  id               uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  call_session_id  uuid REFERENCES call_sessions(id),
  carrier_id       uuid REFERENCES carriers(id),
  rank             integer,
  score            numeric(6,3),
  reason           text,
```

```sql
  ranked_at        timestamptz DEFAULT now()
);

-- Post-call quotes
CREATE TABLE quotes (
  id               uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  call_session_id  uuid REFERENCES call_sessions(id),
  carrier_id       uuid REFERENCES carriers(id),
  product_type     text,
  coverage_amount  integer,
  monthly_premium  numeric(10,2),
  annual_premium   numeric(10,2),
  notes            text
);
```

AGENT MANAGEMENT

SQL — supabase/migrations/003_agents.sql

```sql
CREATE TABLE agencies (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name           text NOT NULL,
  owner_agent_id uuid
);

CREATE TABLE agents (
  id         uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name       text NOT NULL,
  email      text UNIQUE NOT NULL,
  role       text,  -- 'agent' | 'manager' | 'admin'
  agency_id  uuid REFERENCES agencies(id),
  created_at timestamptz DEFAULT now()
);

CREATE TABLE commissions (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_id      uuid REFERENCES agents(id),
  policy_id     uuid REFERENCES policies(id),
  carrier_id    uuid REFERENCES carriers(id),
  amount        numeric(10,2),
  status        text,  -- 'expected' | 'paid' | 'chargedback'
  expected_date date,
  paid_date     date
);

CREATE TABLE leaderboard_snapshots (
  id               uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_id         uuid REFERENCES agents(id),
  period           text,  -- 'daily' | 'weekly' | 'monthly'
  calls_made       integer,
  policies_written integer,
  revenue          numeric(12,2),
  rank             integer
);
```

07 Carrier Knowledge Base Schema

Carrier underwriting rules live as **hand-authored JSON files** in `data/carriers/` and are loaded into Postgres on migration. The ranking engine queries Postgres directly — never the raw JSON at runtime. Start with 5 carriers for the MVP demo.

JSON — data/carriers/mutual_of_omaha.json

```json
{
  "carrier_id": "mutual_of_omaha",
  "name": "Mutual of Omaha",
  "products": ["term_10", "term_20", "term_30", "whole_life", "final_expense"],
  "underwriting_rules": {
    "diabetes": {
      "type_1": { "accepted": false },
      "type_2": {
        "accepted": true,
        "max_a1c": 7.5,
        "min_years_since_diagnosis": 1,
        "excluded_medications": ["insulin_pump"],
        "accepted_medications": ["metformin", "insulin_injection", "glipizide"]
      }
    },
    "tobacco": {
      "current_user": { "accepted": true, "rate_class": "smoker" },
      "quit_years_for_nonsmoker": 3
    },
    "bmi": { "min": 16, "max": 40 },
    "age": { "min": 18, "max": 80 },
    "max_coverage": 500000
  },
  "rate_table": {
    "term_20": {
      "preferred_plus": { "30_male_500k": 25.50 },
      "standard":       { "30_male_500k": 35.00 },
      "smoker":         { "30_male_500k": 85.00 }
    }
  }
}
```

⚠ **Never use RAG to serve ranking decisions.** Carrier rules must be deterministic Postgres queries. RAG may serve agent coaching scripts only, and only in Phase 2+.

08 AI Tool Schema

The Claude API is called via **streaming tool-use** on a background interval (every ~3 seconds) as new transcript chunks arrive. The tool fires only when there is new content since the last extraction. The extracted object is merged into the running underwriting facts panel in Supabase Realtime.

JSON lib/ai/tool-schema.ts → extract_underwriting_facts

```
{
  "name": "extract_underwriting_facts",
  "description": "Extract medical and demographic facts from an insurance call transcript for underwriting evaluation",
  "input_schema": {
    "type": "object",
    "properties": {
      "age":             { "type": "integer" },
      "gender":          { "type": "string", "enum": ["male", "female"] },
      "state":           { "type": "string", "description": "2-letter US state code" },
      "coverage_amount": { "type": "integer", "description": "Requested coverage in dollars" },
      "coverage_type": {
        "type": "string",
        "enum": ["term_10", "term_20", "term_30", "whole_life", "iul", "ul", "final_expense"]
      },
      "conditions": {
        "type": "array",
        "items": {
          "type": "object",
          "properties": {
            "condition": { "type": "string" },
            "details":   { "type": "object" }
          }
        }
      },
      "tobacco_status":   { "type": "string", "enum": ["current", "former", "never"] },
      "tobacco_quit_date": { "type": "string", "format": "date" },
      "medications":      { "type": "array", "items": { "type": "string" } },
      "family_history":   { "type": "array", "items": { "type": "string" } },
      "height_inches":    { "type": "integer" },
      "weight_lbs":       { "type": "integer" }
    }
  }
}
```

09 MVP Demo Scope

The MVP is a **live demo**, not a production system. It must perform a scripted end-to-end walkthrough flawlessly. Every feature listed below must work on demo day with seeded data.

The Demo Script (8 Steps)

1. Agent opens cockpit → sees lead in pipeline board
2. Clicks "Dial" → browser softphone connects to client
3. Client mentions age + coverage amount → carrier shortlist populates
4. Client says *"I have Type 2 diabetes"* → coaching panel injects follow-up questions
5. Client answers A1C, meds, diagnosis date → fact panel updates, carriers re-rank in real-time
6. Client mentions smoking history → another re-rank with animated transition
7. Call ends → quote comparison renders: top 3 carriers with monthly premiums + ranked reasons
8. Agent drags pipeline card → SMS fires to client's phone

3-Week Build Schedule

WEEK 1 — Voice Pipe — Get audio in, transcription out, AI extracting

DAY 1	**Scaffold:** Next.js + Supabase, auth stub, shadcn dark theme, 3-stage pipeline board UI
DAY 2	**Dialer:** Twilio Voice SDK browser softphone — outbound dial, 2-way audio, call lifecycle hooks
DAY 3	**Transcription:** Deepgram streaming STT wired to call audio, live captions rendered in cockpit UI
DAY 4	**AI extraction:** Claude streaming tool-use backend, fact extraction schema, structured facts panel in UI
DAY 5	**Carrier KB:** JSON knowledge base for 5 carriers, unit tests for ranking logic, Postgres migration

WEEK 2 — Moat Layer — The differentiating features that make the demo land

DAY 6	**Coaching panel:** Condition-triggered follow-up questions render contextually during call
DAY 7	**Live re-ranking:** Carrier list animates with Framer Motion transitions + reason tags on each re-rank
DAY 8	**Post-call quote:** Quote comparison screen — top 3 carriers, monthly premiums, ranked reasons
DAY 9	**SMS trigger:** Pipeline stage-change fires Twilio SMS to contact; confirm delivery receipt in UI
DAY 10	**Dashboard shell:** Mock KPI cards, commission ledger, agent leaderboard — seeded with demo data

WEEK 3 — Polish + Demo — Make it bulletproof

DAY 11–12	**Visual polish:** Motion design, typography pass, spacing, color consistency, interaction micro-animations

DAY 13	**Dry run:** End-to-end demo rehearsal with seeded data; fix anything that breaks
DAY 14	**Backup demo:** Record high-quality video walkthrough as fallback if live demo has connectivity issues
DAY 15	**Buffer:** Fix regressions, edge cases, last-minute polish. Do not add new features.

11 Not Building in MVP

Scope control is how the demo ships on time. These are explicitly deferred. Do not add them. If a client asks, say "Phase 2."

- ✕ Real carrier API integrations
- ✕ Commission reconciliation from carrier SFTPs
- ✕ Certificate of Insurance generation
- ✕ SOC2 Type II compliance
- ✕ Full campaign management UI
- ✕ SSO (Microsoft Entra / Google Workspace)
- ✕ Parallel / predictive dialer
- ✕ E-signature / DocuSign
- ✕ Calendar integration (Google/Outlook)
- ✕ Multi-channel inbox
- ✕ Mobile app
- ✕ Carrier SFTP feeds

Compliance Requirements

⚠ **Do not ship any dialer feature without legal review.** TCPA violations carry $500–$1,500 per call in statutory damages. At scale this is existential.

TCPA | TSR | HIPAA-adjacent (GLBA/NAIC) | E&O | Carrier contracts | ACORD | State insurance regs

TCPA

Provable per-number consent required. DNC scrubbing (federal + state) before any dial. Recording disclosures per state. Two-party consent states: CA, FL, IL, MD, MA, MT, NH, PA, WA.

Medical Data (HIPAA-adjacent)

We capture A1C, diagnoses, prescriptions. Not technically HIPAA (insurance = GLBA/NAIC), but treat as PHI. Column-level encryption, audit logs, retention policies required.

E&O — Carrier Ranking

If ranking is wrong and agent relies on it, the brokerage has Errors & Omissions exposure. All ranking logic must be deterministic and auditable. No RAG, no LLM in the ranking path.

State Insurance Regulations

AI "recommending" a carrier must be framed as *agent-selected from AI-ranked options*. The licensed agent is the actor. The AI is a tool. This distinction must be in the UI copy.

Carrier Contracts

Redistribution of carrier feed data may be restricted by contract. Get legal signoff before connecting carrier SFTPs in Phase 2.

ACORD Forms

ACORD forms are licensed IP. We cannot generate Certificates of Insurance without rights. Deferred entirely.

RAG / Knowledge Architecture

Real-time Ranking — NO RAG

Deterministic TypeScript function querying structured Postgres tables. Sub-100ms. Auditable. No vector search in this path, ever.

Phase 2 — Sales RAG

pgvector in Supabase for: sales scripts library, objection handling, historical call transcript search ("similar diabetic cases that closed").

Phase 3 — Carrier Guide Ingestion

RAG pipeline over 300-page carrier underwriting PDFs → human-reviewed extraction → structured rules tables. RAG feeds the DB; never serves agents directly.

→ **No Neo4j.** Postgres with foreign keys is the knowledge graph for this product. pgvector handles semantic search when we need it.

14 Key Decisions Already Made

These decisions are finalized. Do not re-litigate them during the MVP build. Raise concerns in a retrospective after the demo.

1. **Deterministic rules for underwriting — NOT RAG**
   Auditable, fast (<100ms), E&O-defensible. RAG hallucination in a ranking context creates legal liability.

2. **Supabase over custom backend**
   RLS, Realtime, pgvector, Edge Functions, Auth all in one platform. Fewer moving parts for a 3-week build.

3. **Browser softphone (Twilio Voice SDK) over hardware integration**
   Simpler architecture, demo-able without physical setup, works in any browser. Hardware integration is Phase 2+.

4. **Claude tool-use for extraction — not a fine-tuned model**
   Flexibility to iterate on the schema without retraining. No training pipeline to maintain. Tool-use is production-grade.

5. **Dark theme, insurance-professional aesthetic**
   The buyer is an insurance brokerage, not a startup. Design signals trust and authority, not playfulness.

6. **Internal tool first, product later**
   Dramatically reduces compliance surface during MVP. No external user registration, no multi-tenant billing, no consumer data exposure.

7. **Single-line dialer first**
   Parallel dialing requires robust TCPA consent architecture and DNC infrastructure. Deferred to prevent legal exposure on day one.

Reference Products

LeadChampPro

GHL skin for insurance. This is what the client uses today. We have login access to study their workflows.

STUDY + REPLACE

GoHighLevel

White-label CRM. The pipeline/workflow/SMS engine is the reference implementation for our pipeline UX.

DESIGN REFERENCE

Cluely

Horizontal AI sales coaching. Proves market demand. We beat it on vertical integration — they're generic, we're surgical.

BEAT IT

Applied Epic / Vertafore AMS360

Enterprise AMS platforms. We are NOT competing here. We complement or replace for smaller brokerages.

COMPLEMENT

NowCerts / EZLynx / QQCatalyst

Mid-market AMS. Closest feature competitors, but none have an AI dialer coaching layer.

WE HAVE THE MOAT

Kixie

Power dialer with parallel dialing. Reference implementation for Phase 2 parallel dialer UX and TCPA architecture.

PHASE 2 REFERENCE

Team & Roles

Z

Zazu — Technical Co-founder / AI + Product

Agentic Engineering Lead

Dialer architecture, transcription pipeline, Claude orchestration, product vision, engineering direction

BE

Senior Backend Engineer — TBD

Hiring / Onboarding

Carrier integrations, multi-AMS sync, SOC2 architecture (Phase 2)

FE

Senior Frontend Engineer — TBD

Hiring / Onboarding

GHL-level UI polish, real-time dashboards, dialer cockpit UI, Framer Motion animations

CA

Compliance Advisor — Fractional

~$3–5K/mo engagement

TCPA, state insurance regulations, PHI handling, DNC infrastructure, E&O risk review

Local Setup

BASH Terminal

```bash
# Clone and install
git clone [repo-url]
cd nexus-os
npm install

# Start Supabase locally
npx supabase start
npx supabase db push

# Dev server
npm run dev
```

REQUIRED ENVIRONMENT VARIABLES

```
NEXT_PUBLIC_SUPABASE_URL=your-project-url
NEXT_PUBLIC_SUPABASE_ANON_KEY=anon-key
SUPABASE_SERVICE_ROLE_KEY=service-role-key
TWILIO_ACCOUNT_SID=ACxxxxxxxx
TWILIO_AUTH_TOKEN=auth-token
TWILIO_PHONE_NUMBER=+1xxxxxxxxxx
DEEPGRAM_API_KEY=dg-key
ANTHROPIC_API_KEY=sk-ant-key
```

File Structure

TREE nexus-os/

```
nexus-os/
├─ app/                          # Next.js app router
│  ├─ (auth)/                    # Login / signup pages
│  ├─ (dashboard)/               # Main app shell (requires auth)
│  │  ├─ pipeline/               # Drag-drop pipeline board
│  │  ├─ cockpit/                # AI call cockpit (the moat)
│  │  ├─ contacts/               # Contact list + detail views
│  │  ├─ dashboard/              # KPIs, commissions, leaderboard
│  │  ├─ campaigns/              # Lead list management (stub)
│  │  └─ settings/               # Agent / agency settings
│  └─ api/                        # API routes
│     ├─ twilio/                  # Voice webhooks (TwiML, status)
│     ├─ deepgram/                # STT proxy (auth'd WebSocket)
│     ├─ ai/                      # Claude extraction endpoint
│     └─ carriers/                # Ranking + quoting endpoints
├─ components/
│  ├─ pipeline/                   # Board, cards, stage columns, drag
│  ├─ cockpit/                    # Transcript, facts panel, coaching, ranking
│  ├─ dashboard/                  # Charts, KPI cards, leaderboard
│  └─ ui/                         # shadcn base components
├─ lib/
│  ├─ carriers/                   # Rules engine, ranking fn, quote generator
│  ├─ twilio/                     # Voice SDK setup, token generation
│  ├─ deepgram/                   # Streaming STT client, utterance buffer
│  ├─ ai/                         # Claude tool-use client, extraction loop
│  ├─ supabase/                   # Browser + server clients, RLS helpers
```

```
│   └─ utils/                    # Shared utilities
├─ supabase/
│   └─ migrations/               # SQL migration files (numbered)
├─ data/
│   └─ carriers/                 # JSON carrier knowledge base (5 files)
├─ package.json
└─ .env.local                    # Never commit this
```

→ **File size rules:** Keep all files under 400 lines. 800 lines is the hard ceiling. If a file approaches 400 lines, split it by feature subdomain before continuing.